AllMinds

Backend, Database & Authentication

Express REST API, PostgreSQL data model, how users are stored and logged in, and how the backend connects the frontend to the AI agents

Neurodivergent-affirming AI teaching companion — HackAstone Agentic AI

Generated from the source of the HackAstone-Agentic-AI/AllMinds repository

1. Role of the backend in the overall goal

The Express service (apps/backend, port 5000) is the **system of record and the security gateway**. It owns user identity, every piece of classroom data (learners, progress, schedules, saved plans, letters) and the third-party integrations that need secrets (OAuth, Google Calendar, SMTP). It is also the only door to the AI service: every AI request is authenticated and rate-limited here before it reaches FastAPI. That lets the agents stay stateless and focus purely on generating teaching support.

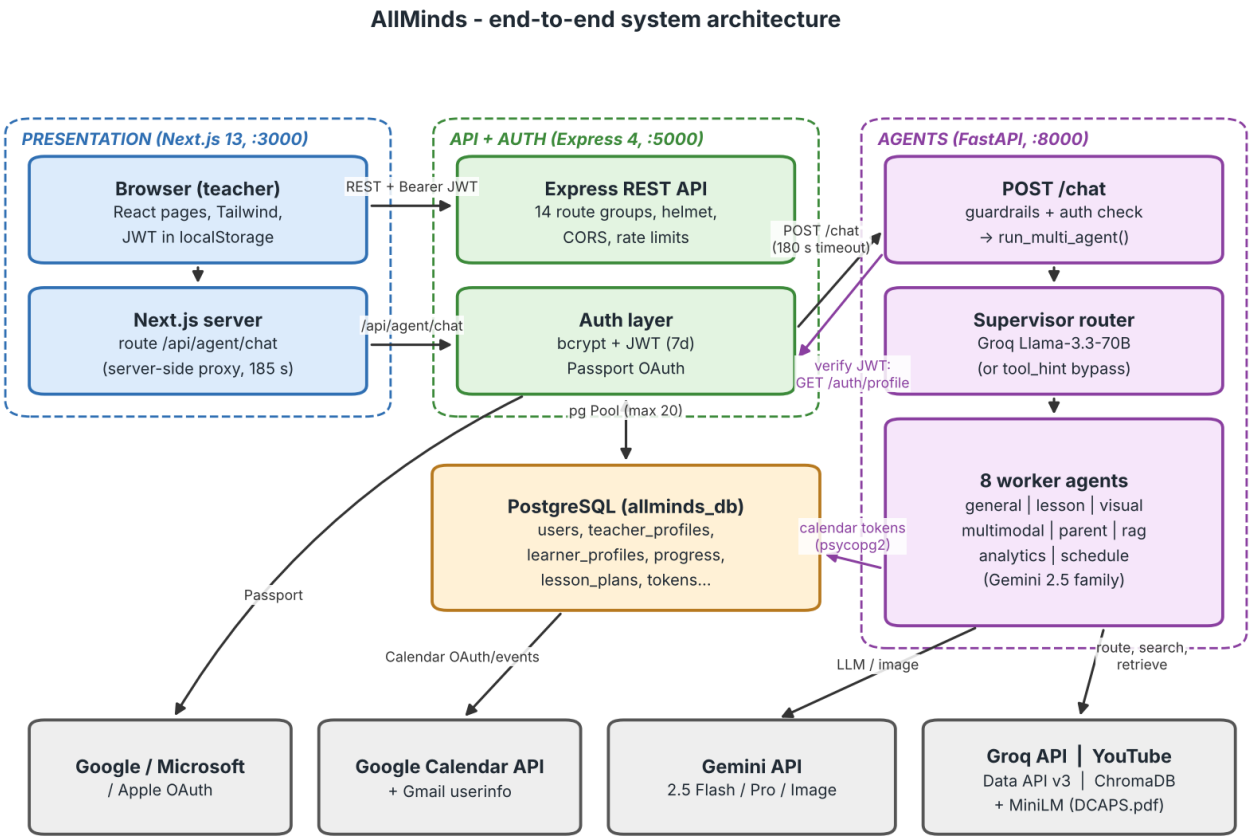


Figure 1. Position of the backend (green) between the frontend and the agents.

1.1 Stack

Area	Implementation
Runtime / framework	Node 18+, Express 4.18 (server.js boots src/app.js)
Database driver	pg (node-postgres) Pool: max 20 connections, idle 30 s, connect timeout 2 s; DATABASE_URL with SSL in production, or DB_HOST/PORT/USER/PASSWORD/NAME locally
Security	helmet, CORS allow-list, express-rate-limit, bcrypt, jsonwebtoken, crypto (SHA-256, HMAC)
Auth providers	passport, passport-google-oauth20, passport-oauth2 (Microsoft, Apple with JWKS check)
Other	multer (admin document upload), nodemailer (reset/verify e-mail), compression
Schema management	npm run db:init (docs/database/allminds_complete_schema.sql) then npm run db:migrate (ordered .sql files, tracked in schema_migrations, each in a transaction)
Deployment	Render web service (Frankfurt) with managed Postgres; preDeploy runs init + migrate; health check GET /api/health

Table 1. Backend stack

2. Request pipeline and API surface

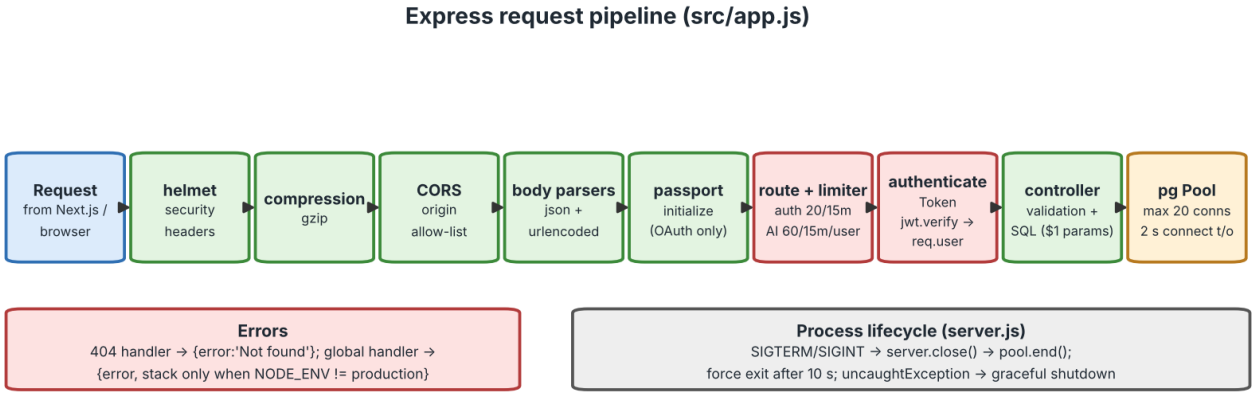


Figure 2. Every request passes the same middleware chain before reaching a controller.

Mount point	Endpoints	Protection
/api/auth	POST /register, POST /login, GET /profile, GET|POST /verify-email, POST /resend-verification	login/register: 20 req / 15 min / IP
/api/auth (oauthRoutes)	GET /google, /microsoft, /apple (+ /callback), POST /oauth/exchange	30 req / 15 min
/api/auth (passwordReset)	POST /forgot-password, POST /reset-password	5 / hour and 20 / hour
/api/profile	GET /, PUT / (users + teacher_profiles + language_pref)	JWT
/api/learners	GET /, POST /, POST /bulk (1-1000 rows, one transaction), GET|POST /:id/sensory-logs, GET|POST /:id/progress	JWT, scoped by teacher_id
/api/activities	GET /activities, lesson-plans CRUD, tasks GET/POST/PATCH, schedule-items GET/POST/PATCH/DELETE	JWT
/api/parent-communications	GET /?learnerId, POST /	JWT
/api/forum	GET /, POST /, GET|POST /:postId/comments	GET public, POST JWT
/api/strategies	GET /, GET /:id, POST /, POST /:id/vote (UNIQUE strategy_id,user_id)	writes JWT
/api/ngos	GET /, GET /:id, POST /:id/review	review JWT
/api/school	classes, teachers, assign-teacher, learners, documents/upload (multer)	JWT + authorizeAdmin
/api/stats	GET /admin, GET /teacher	JWT
/api/calendar	GET /connect, GET /callback, GET /status, POST /events, DELETE /disconnect	JWT (callback uses signed state)
/api/agent	POST /chat -> FastAPI /chat	JWT + 60 req / 15 min per user
/api/health	status, timestamp, configured OAuth providers	public

Table 2. Complete Express route map

3. Database - how users and data are stored

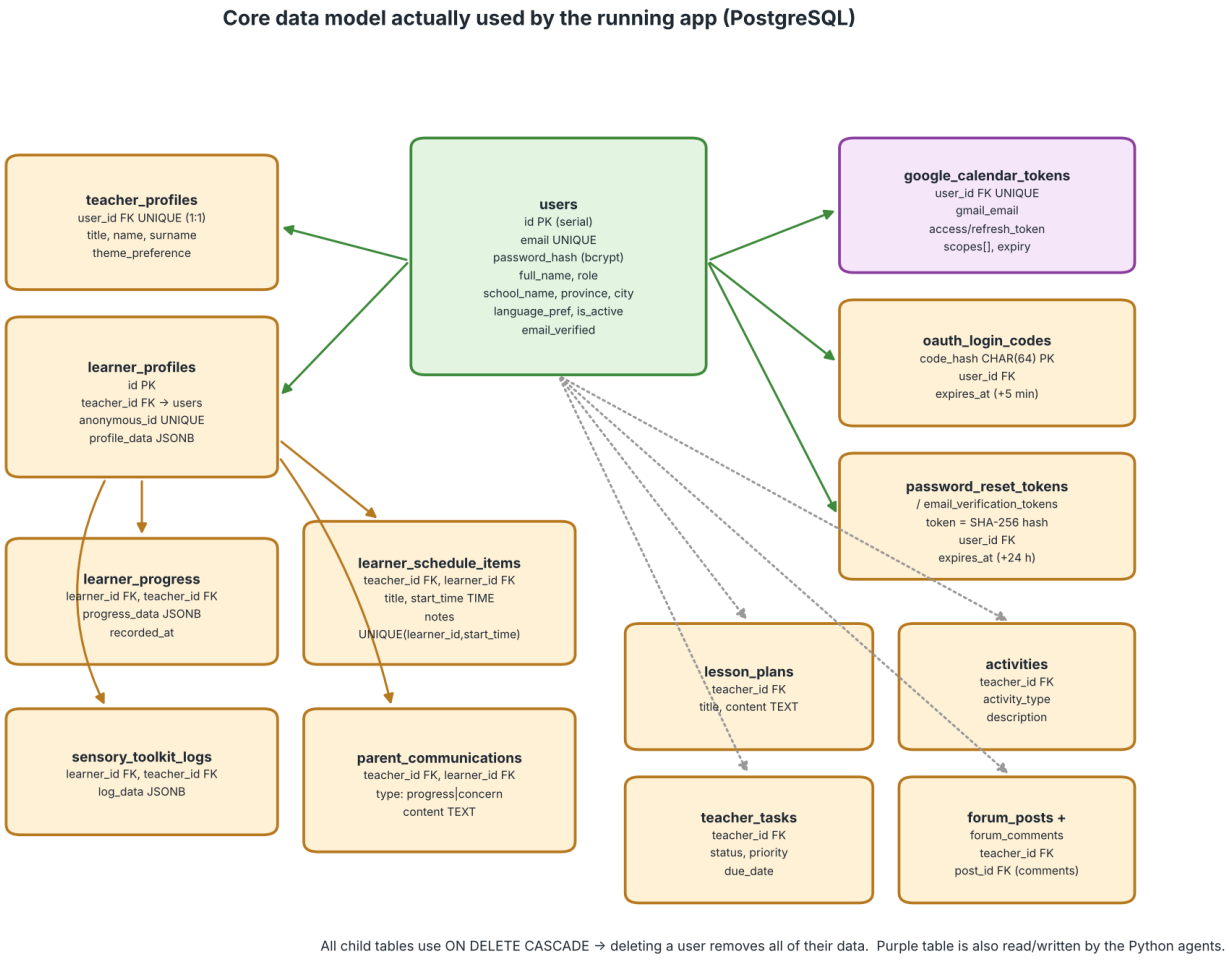


Figure 3. Entity-relationship view of the tables the controllers query. users is the root of everything; learner data hangs off learner_profiles.

3.1 How a user is stored

- **users** holds identity: unique lower-cased e-mail, **bcrypt hash** (never the password), full name, role ('teacher' or 'admin'), school/province/city/phone, language_pref, is_active and email_verified. A trigger keeps updated_at current.
- **teacher_profiles** is a strict 1:1 extension (UNIQUE user_id) with title, name, surname and theme_preference. Register and OAuth both upsert it, so every user always has one.
- OAuth-only accounts are inserted with *password_hash = ''*; bcrypt.compare can never match an empty hash, so those accounts cannot be logged into with a password - only through their provider.
- Secrets that are bearer-equivalent are stored **hashed**: password-reset and e-mail verification tokens (SHA-256, 24 h expiry, single use), and OAuth exchange codes (SHA-256, 5 min, deleted on use).

3.2 How learners are stored (privacy by design)

Learners are not users. A **learner_profiles** row belongs to one teacher (teacher_id) and is identified by an **anonymous_id** (e.g. LEARNER-1A2B3C4D generated with crypto.randomUUID when the CSV lacks one). Everything else lives in a flexible **profile_data JSONB** column, so schools can upload any CSV shape. Progress and sensory observations are append-only JSONB rows in learner_progress and sensory_toolkit_logs; each insert also writes an *activities* row that feeds the dashboard timeline. Every query filters by *teacher_id = req.user.id*, so one teacher can never read another's learners.

> The schema file also contains a school-level model (schools, classes, teacher_classes, learners, curriculum_documents, lesson_history, ngos, community_strategies) used by the admin /api/school and /api/stats endpoints. The teacher app primarily uses the learner_profiles family shown in the figure.

4. Authentication and login

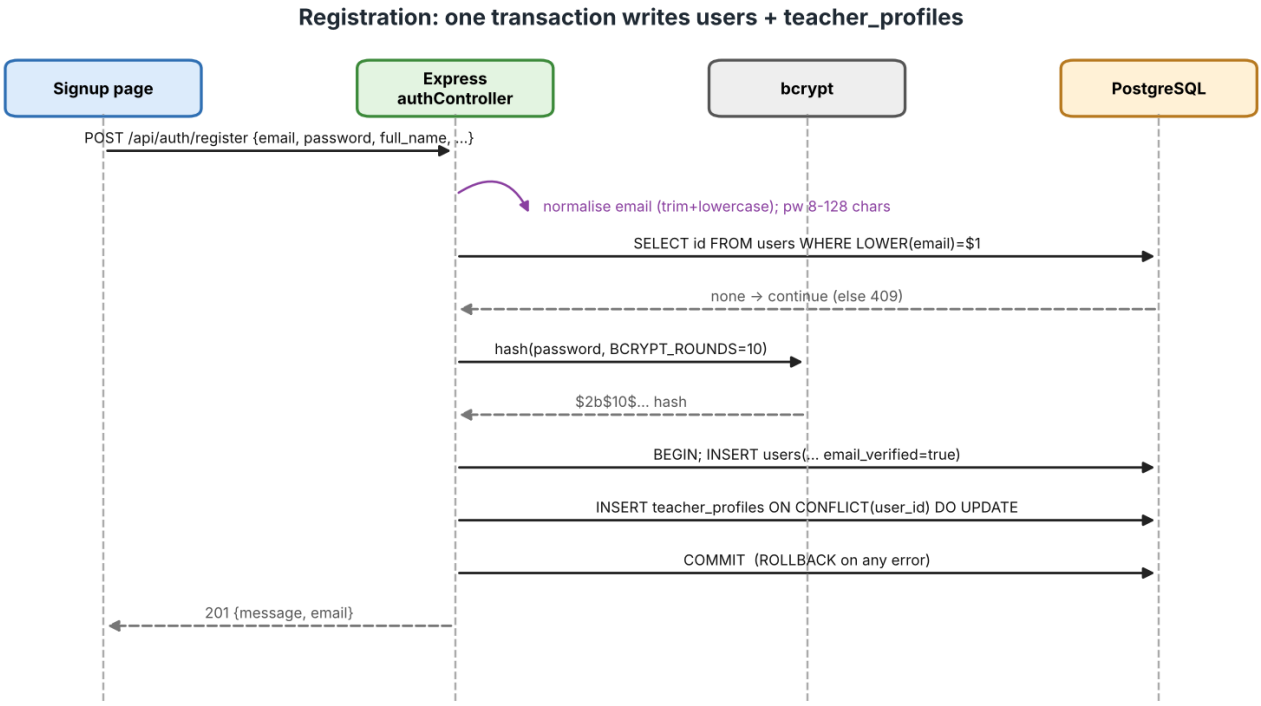


Figure 4. Registration writes the user and profile atomically.

4.1 Login and the JWT

POST /api/auth/login normalises the e-mail, loads the user, rejects inactive accounts, runs *bcrypt.compare* and issues *jwt.sign({id, email, role}, JWT_SECRET, {expiresIn: '7d'})*. The same generic 'Invalid email or password' is returned for unknown users and bad passwords (no account enumeration). **authenticateToken** reads 'Authorization: Bearer', verifies the signature and expiry, and sets *req.user*; expired -> 401 'Token expired', tampered -> 403. **authorizeAdmin** additionally requires role 'admin'. The token is stateless, which is why the Python agents can validate it simply by calling GET /api/auth/profile.

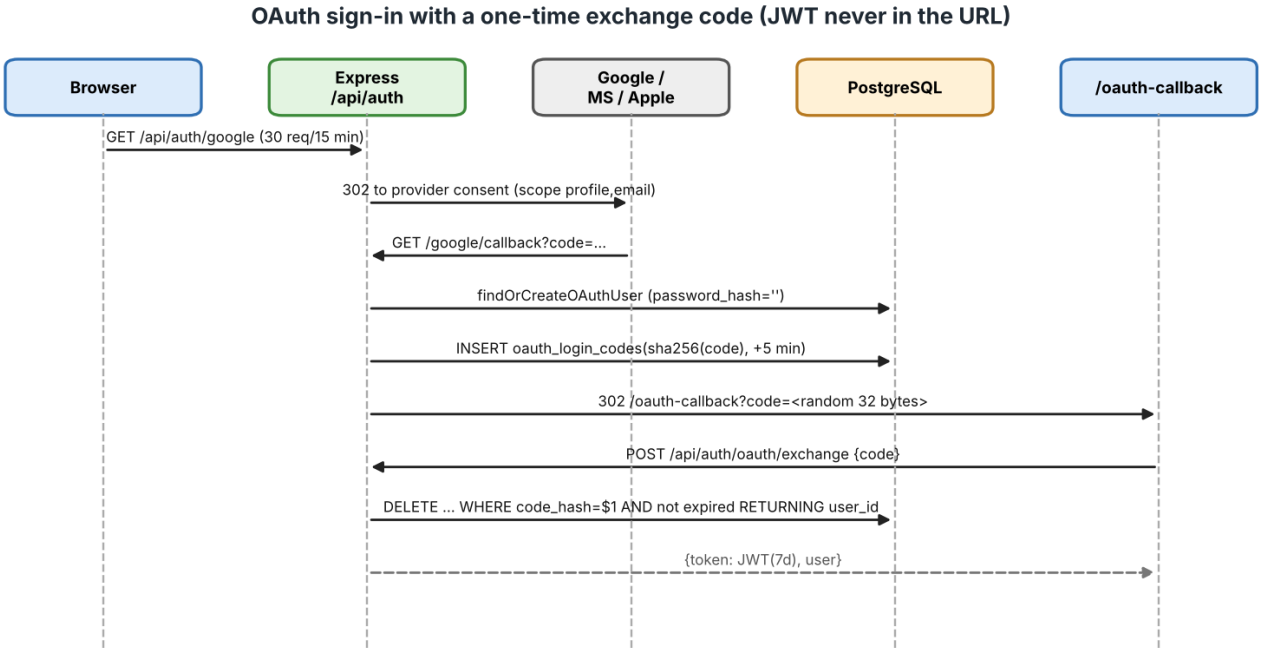


Figure 5. OAuth flow for Google / Microsoft / Apple.

4.2 Password reset and e-mail verification

forgot-password always answers with the same message (no enumeration), stores SHA-256(token) with a 24 h expiry, removes older tokens for that user and e-mails *FRONTEND_URL/reset-password?token=...*. In development without SMTP it returns reset_link in the JSON. reset-password looks up the hash, checks expiry, deletes the token (single use) and stores a new bcrypt hash. E-mail verification uses the same pattern but is currently switched off: register sets email_verified=true until SMTP is configured.

4.3 Google Calendar connection

GET /api/calendar/connect builds Google's consent URL with a **signed state**: base64url(userId:timestamp:HMAC-SHA256(JWT_SECRET)). The callback verifies the HMAC with timingSafeEqual, rejects states older than 10 minutes, exchanges the code (10 s HTTPS timeout), fetches the Gmail address and upserts **google_calendar_tokens** (keeping the old refresh token if Google does not send a new one). POST /api/calendar/events refreshes the access token if it expires within 60 s and creates the event in Africa/Johannesburg time.

5. How the backend connects the frontend and the agents

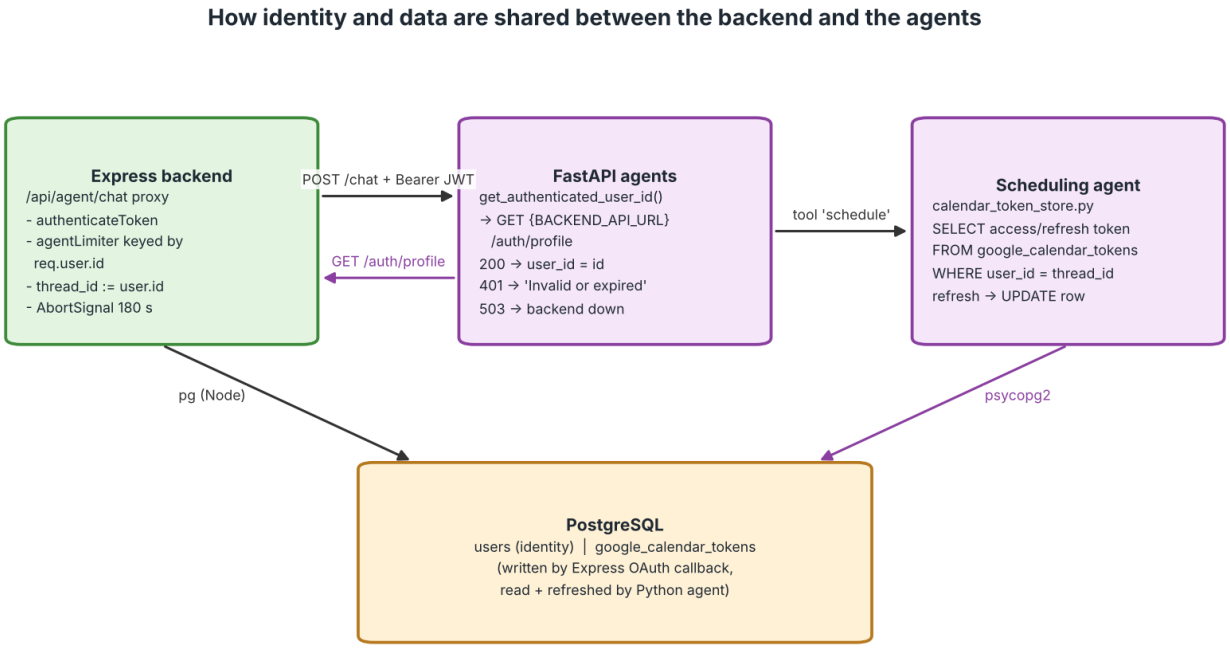


Figure 6. Two integration paths: the HTTP proxy for every AI request, and the shared calendar token table used by the scheduling agent.

- **Proxy:** agentController.proxyAgentChat copies the request body, forces *thread_id* to the authenticated user's id (a client cannot impersonate another thread), forwards the Authorization header and maps upstream errors ({detail} from FastAPI) to {error}. Timeout -> 502 'The AI request timed out'.
- **Identity check-back:** FastAPI refuses any /chat without a Bearer token and validates it against Express (30 s timeout). The agents never hold JWT_SECRET.
- **Shared table:** the scheduling agent reads and refreshes tokens in google_calendar_tokens directly through psycopg2 using the same DATABASE_URL - Express writes, Python reads.
- **Persisting AI output:** agents return text only; the frontend saves results through normal REST endpoints (lesson_plans, parent_communications), so all durable data flows through Express.

5.1 Backend latency budget

Operation	Typical time (estimate)	Configured limit
Simple SELECT via pool (same region)	2-20 ms	2 s connection timeout
Login (bcrypt cost 10)	60-150 ms (bcrypt dominates, ~50-100 ms)	20 req / 15 min / IP
Bulk learner upload (1000 rows, one transaction)	0.5-3 s	1000 rows max
Calendar token exchange / event create	300 ms - 1.5 s (Google round trips)	10 s per HTTPS call
AI proxy	adds ~5-30 ms; total dominated by the agent (see PDF 3)	180 s upstream timeout

Table 3. Backend timings. Estimates assume services co-located in one Render region; the repository has no load-test data.

6. Hackathon final - likely questions and model answers

Q1. How are passwords stored?

Only as bcrypt hashes (cost factor BCRYPT_ROUNDS, default 10) in users.password_hash. Login uses bcrypt.compare; passwords must be 8-128 characters. OAuth users have an empty hash so password login is impossible for them.

Q2. Why JWT instead of server sessions?

We have three services. A stateless signed token lets Express validate locally and lets FastAPI validate by asking Express (/auth/profile) without shared session storage. Trade-off: revocation is hard before the 7-day expiry - mitigations are short-lived tokens plus refresh tokens or a deny-list.

Q3. How do you stop one teacher seeing another teacher's learners?

Every learner, progress, sensory-log, schedule, plan and letter query is filtered by teacher_id = req.user.id taken from the verified JWT, never from the request body. Foreign keys with ON DELETE CASCADE keep ownership consistent.

Q4. How is learner data protected under POPIA?

Learners are stored with an anonymous_id and teacher-controlled JSONB data rather than as user accounts; AI requests are PII-scrubbed (ID numbers, e-mails, phones) before reaching an LLM; tokens are hashed; CORS is restricted; Helmet sets security headers. Next steps: encryption at rest for calendar tokens and a data-retention policy.

Q5. Why JSONB for profile_data and progress_data?

Schools upload CSVs with different columns and progress observations evolve. JSONB keeps the schema stable while allowing flexible fields, and Postgres can still index and query inside it.

Q6. Explain the OAuth exchange-code design.

After the provider callback we never put the JWT in a URL. We redirect with a random 32-byte code, store only its SHA-256 with a 5-minute expiry, and the frontend swaps it for a JWT via POST. The DELETE ... RETURNING makes the code single-use even under races.

Q7. What rate limits exist and why?

Login/register 20 per 15 min per IP (credential stuffing), OAuth 30 per 15 min, forgot-password 5 per hour (e-mail abuse), reset 20 per hour (token guessing), AI chat 60 per 15 min per user (LLM cost control).

Q8. How do schema changes get deployed?

Render's preDeploy runs db:init then db:migrate. The migration runner applies .sql files in filename order, each inside a transaction, and records them in schema_migrations so they run exactly once.

Q9. What happens if the agent service is down?

The proxy catches the fetch failure and returns 502 'Unable to reach the agent service'; the rest of the app (learners, plans, forum) is unaffected because it does not depend on the agents.

Q10. What are the known weaknesses?

E-mail verification is disabled until SMTP is configured; JWTs live in localStorage; Google calendar access/refresh tokens are stored in plaintext; the schema SQL and migrations overlap (two sources of truth); there are no automated backend tests in the repo. Each has a clear remediation path.